

```
from google.colab import drive
drive.mount('/content/drive')
```

 Mounted at /content/drive

```
import os
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import cross_val_score
from google.colab import drive
from PIL import Image # Import Image from PIL to open and process images

# Mount Google Drive
drive.mount('/content/drive')

# Define the path to your dataset
image_folder_path = "/content/drive/MyDrive/output/train"

# Define folder-to-label mapping
folder_label_mapping = {'CN Top View': 0, 'AD Top View': 1, 'MCI Top View': 2}

# Initialize lists for images and labels
resized_images = []
labels = []

# Load images and assign labels
for folder, label in folder_label_mapping.items():
    folder_path = os.path.join(image_folder_path, folder)
    for image_file in os.listdir(folder_path):
        image_path = os.path.join(folder_path, image_file)
        img = Image.open(image_path).resize((224, 224)) # Resize to (224, 224)
        resized_images.append(np.array(img))
        labels.append(label)

# Convert labels to numpy array
labels = np.array(labels)

# Convert resized_images to numpy array
resized_images = np.array(resized_images)

# Flatten the images to 1D arrays for SVM
X = resized_images.reshape(resized_images.shape[0], -1) # Flattening to (num_samples, num_features)

# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.2, random_state=42)

# Standardize the images (important for SVM)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Create and train the SVM model
svm_model = SVC(kernel='rbf', random_state=42)
svm_model.fit(X_train_scaled, y_train)

# Predict on the test set
y_pred = svm_model.predict(X_test_scaled)

# Calculate accuracy and classification report
accuracy = accuracy_score(y_test, y_pred)
print(f"SVM Test Accuracy: {accuracy:.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Perform Cross-Validation for more robust accuracy estimation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cross_val_accuracy = cross_val_score(svm_model, X, labels, cv=cv, scoring='accuracy')

print(f"Stratified Cross-Validation Accuracy: {cross_val_accuracy.mean():.2f}")

# Optionally, you can try different kernels (e.g., linear or polynomial) and tune hyperparameters
```

optionally, you can try different kernels (e.g., linear or polynomial) and tune hyperparameters.

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
SVM Test Accuracy: 0.99

```
Classification Report:
              precision    recall  f1-score   support

     0       1.00      0.98      0.99       213
     1       1.00      0.99      0.99        95
     2       0.98      1.00      0.99       239

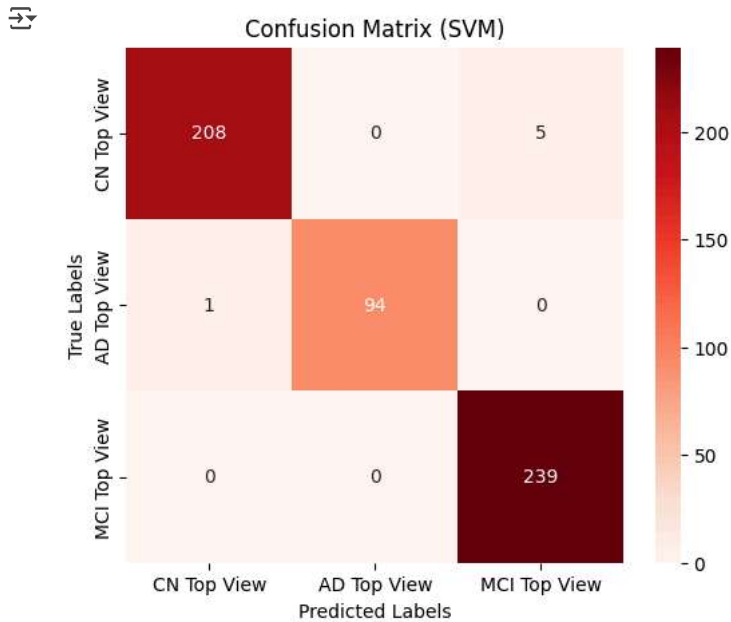
 accuracy          0.99
 macro avg          0.99
weighted avg          0.99
```

Stratified Cross-Validation Accuracy: 0.98

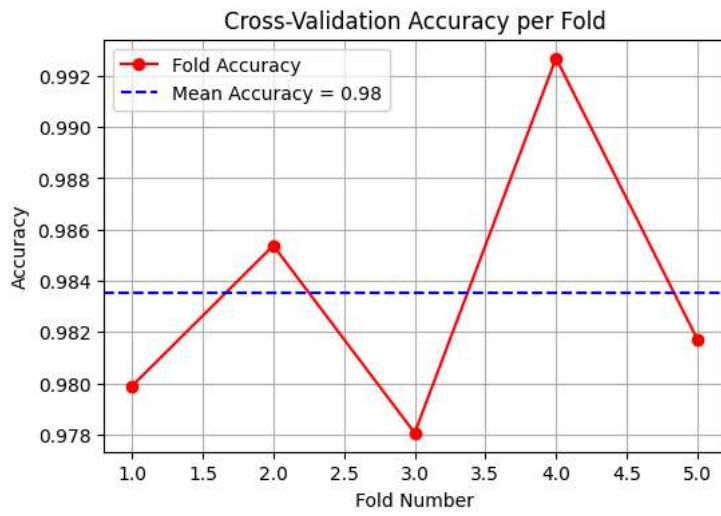
```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# --- Confusion Matrix ---
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap="Reds", xticklabels=folder_label_mapping.keys(), yticklabels=folder_label_mapping.keys())
plt.xlabel("Predicted Labels")
plt.ylabel("True Labels")
plt.title("Confusion Matrix (SVM)")
plt.show()
```



```
plt.figure(figsize=(6, 4))
plt.plot(range(1, len(cross_val_accuracy)+1), cross_val_accuracy, marker='o', color='red', label="Fold Accuracy")
plt.axhline(y=cross_val_accuracy.mean(), color='blue', linestyle='--', label=f"Mean Accuracy = {cross_val_accuracy.mean():.2f}")
plt.xlabel("Fold Number")
plt.ylabel("Accuracy")
plt.title("Cross-Validation Accuracy per Fold")
plt.legend()
plt.grid(True)
plt.show()
```



Start coding or [generate](#) with AI.